

Arquitetura e Decisões Técnicas

Resumo executivo do documento de arquitetura da plataforma inteligente de gestão de frotas: stack, multi-tenant, dados e telemetria, inteligência artificial, infraestrutura, segurança e custos — acrescido do que já está efetivamente implementado na Fase 1 do produto.

PRODUTO
RookHub

DOCUMENTO DE ORIGEM
Arquitetura e Decisões Técnicas v1.0 (20 páginas)

BASE
PRD RookHub 1.0 · DESIGN.md · Visão e Escopo de Negócio

FASE DO PRODUTO
Fase 1 — MVP frontend, sem backend

PÚBLICO
Engenharia, Dados, Infraestrutura e Produto

1 Sumário executivo

O RookHub é um SaaS B2B multi-tenant para gestão de frotas de transportadoras rodoviárias de carga. A arquitetura precisa atender, ao mesmo tempo, a seis exigências que raramente convivem: alta densidade informacional com identidade visual sofisticada, operação offline integral no aplicativo do motorista, ingestão contínua de telemetria (7 a 14 milhões de pontos por mês por cliente), isolamento multi-tenant rigoroso, assistente de IA generativa sem qualquer vazamento de dado por RBAC e custo operacional compatível com o estágio pré-escala.

A solução escolhida é um monólito modular em Java/Spring Boot com worker dedicado, sobre PostgreSQL + TimescaleDB, servindo três aplicações frontend independentes em monorepo, hospedado em AWS ECS Fargate, com a IA orquestrada por LangChain4j sobre o Google Gemini.

PRINCÍPIO CONDUTOR

Maximizar a velocidade de entrega de regra de negócio (148 regras no PRD) e minimizar a superfície operacional (time de 3 desenvolvedores de backend), sem comprometer o isolamento de dados — o requisito não-negociável do produto.

Como as decisões são identificadas

- FE-xx — Frontend · BE-xx — Backend · DAT-xx — Dados e armazenamento
- IA-xx — Inteligência artificial · INF-xx — Infraestrutura e DevOps · INT-xx — Integrações
- RT-xx — Risco técnico · DT-xx — Dívida técnica assumida conscientemente
- Toda decisão é rastreável ao requisito do PRD que a motivou, e todas foram aprovadas com alternativas, prós, contras e impacto financeiro.

2 Stack de referência

CAMADA	TECNOLOGIA
Painel web	Vite + React 19 + React Router (SPA, 100% autenticada)
PWA do motorista	Vite + React 19 + Workbox + Dexie (IndexedDB), offline-first
Site institucional	Next.js 15 (SSR/SSG, SEO e página de planos)
Estilização	Tailwind CSS v4 + shadcn/ui + tokens do DESIGN.md
Gráficos e mapas	visx (Painel do Dono) + Recharts (demais) · Mapbox GL JS
Tempo real	WebSocket (STOMP sobre SockJS) + Redis pub/sub
Backend	Java 21 LTS + Spring Boot 3.x + Spring Modulith (13 módulos)
API	REST + OpenAPI 3.1, com cliente TypeScript gerado
Persistência	Spring Data JPA (CRUD) + jOOQ (toda consulta analítica)
Banco principal	PostgreSQL 16 + TimescaleDB (Timescale Cloud sobre AWS, sa-east-1)
Cache, filas e locks	Redis (AWS ElastiCache) + Redisson · Spring Batch para lote
Object storage	Cloudflare R2 (egress gratuito, API compatível com S3)
Autenticação	Spring Security — JWT + refresh rotativo, Argon2id
IA e voz	LangChain4j + Google Gemini · Google STT + ElevenLabs TTS
Computação e IaC	AWS ECS Fargate + ALB · Terraform/OpenTofu · GitHub Actions
Observabilidade	Datadog (APM e logs) + Sentry, instrumentação OpenTelemetry

3 Frontend, mobile e design system

FE-03 / FE-04 / FE-05 — Três aplicações, um monorepo

O painel é uma SPA Vite porque é 100% autenticado e não precisa de SEO; o site institucional é Next.js por causa de SEO, SSG e autonomia do marketing. O PWA do motorista é aplicação separada, com bundle, service worker e deploy próprios: os requisitos são antagônicos aos do painel — glassmorphism contra legibilidade sob sol, banda larga contra 3G em rodovia, offline integral e alvos de toque grandes. Um bundle único faria o motorista baixar gráficos, mapas e IA que jamais usaria.

FE-06 / FE-07 — Vidro com rede de proteção

Tailwind v4 (motor Oxide, configuração via @theme em CSS) com shadcn/ui sobre Radix, e primitivos próprios em packages/ui (GlassCard, GlassPanel, DataTable, FleetIndicator). Como o backdrop-filter: blur() é uma das operações mais caras do compositor, há fallback automático para superfície opaca, desativação global em hardware modesto, proibição de blur por item em listas longas e gate de contraste WCAG AA com axe-core bloqueando o build.

FE-08 — Estado e camada offline

CAMADA	ESCOLHA
Server state	TanStack Query v5 — cache, retry, atualização otimista, invalidação
Client state	Zustand — leve, sem boilerplate
Offline	Dexie.js (IndexedDB) + fila de sincronização própria
Service worker	Workbox — precache do shell e Background Sync para fotos

A fila é própria porque as regras do PRD são específicas demais para uma engine genérica: dois timestamps distintos (dispositivo e servidor), prioridade de dados estruturados sobre fotos, servidor como autoridade em conflito e capacidade dimensionada em 7 dias. Todo registro criado offline carrega um client_uuid (UUID v7) usado como chave de idempotência.

FE-09 a FE-12 — Gráficos, mapas, tempo real e paleta

- Regra de fronteira dos gráficos: visx apenas em widget fixo e desenhado (Painel do Dono); qualquer gráfico gerado por dados dinâmicos usa Recharts.
- Mapbox GL JS com mitigações obrigatórias de custo — instância única de mapa por sessão, atualização por setData e migração para MapLibre acima de US\$100/mês (RT-02).
- WebSocket com STOMP eliminou serverless do backend, exigiu Redis pub/sub e sticky sessions no ALB; o JWT é validado no handshake e a inscrição em tópico de outro tenant é rejeitada.
- Paleta gerada em rampa OKLCH a partir de três âncoras imutáveis: Indigo #6366F1, Cyan #06B6D4 e Midnight #0B1220, com matiz azulado preservado em toda a rampa neutra.
- Semânticos escolhidos deliberadamente na faixa clara de cada matiz: vermelho escuro convencional reprova em WCAG AA sobre superfície translúcida escura.

4 Backend, APIs e multi-tenant

Java 21 LTS com Spring Boot 3.x e Spring Modulith, que impõe as fronteiras entre os módulos em tempo de build — se costs importar uma classe interna de safety, o teste de arquitetura falha. Programação reativa foi descartada: com 3 desenvolvedores e 148 regras de negócio, o custo de depurar stack traces reativos não se paga; virtual threads do Java 21 tornam o modelo bloqueante viável. Microserviços também foram descartados, e o worker roda a mesma imagem Docker com outro profile Spring, isolando o processamento pesado do processo que atende requisições.

BE-14 — Isolamento multi-tenant: o requisito mais crítico

O PostgreSQL RLS lê o tenant de uma variável de sessão. Com connection pooling, se a variável for definida com SET em vez de SET LOCAL, a conexão volta ao pool carregando o tenant anterior e a próxima requisição lê dados de outro cliente. Esse defeito não aparece em teste unitário — manifesta-se apenas sob concorrência, em produção.

```
CREATE POLICY tenant_isolation ON vehicles
  USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

// ponto único de acesso, sempre no escopo da transação
jdbc.update("SELECT set_config('app.current_tenant_id', ?, true)", tenantId);
```

CAMADA DE GARANTIA	MECANISMO
Runtime	Aborta qualquer transação iniciada sem tenant definido, exceto rotas marcadas como cross-tenant
Compilação	Teste ArchUnit falha o build se DataSource, EntityManager ou DSLContext forem injetados fora do TenantContext
Schema (CI)	Job varre o information_schema e falha o build se alguma tabela de domínio não tiver tenant_id NOT NULL e policy RLS — bloqueante e não desativável por decisão de squad
Concorrência	Teste de integração executa dois tenants em paralelo sobre o mesmo pool e asseve ausência de vazamento

Acesso cross-tenant é exclusivo do SUPER_ADMIN, exige motivo textual e sempre grava no log de auditoria. O controle de acesso tem duas camadas independentes verificadas no backend: o plano contratado inclui o módulo (@RequiresEntitlement) e o papel pode executar a ação (@PreAuthorize).

BE-04 / BE-10 / BE-11 — API, persistência e autenticação

- REST com OpenAPI 3.1 gerado do código; GraphQL foi descartado por segurança, não por preferência — autorização por campo em um grafo é muito mais difícil de auditar do que por função nomeada.
- Persistência híbrida: JPA no CRUD, jOOQ obrigatório em toda consulta analítica e em todas as funções consumidas pelo assistente de IA.
- Autenticação própria com Spring Security: provedores gerenciados cobram por usuário ativo e cada cliente traz dezenas de motoristas — além disso o acesso do motorista (QR + PIN) é não-padrão.
- Rotação de refresh token com detecção de reuso: token já utilizado que reaparece revoga toda a família de tokens do usuário.

5 Dados, telemetria e armazenamento

O banco é PostgreSQL 16 com TimescaleDB no Timescale Cloud, na mesma região AWS da aplicação: o RDS foi descartado porque a AWS não oferece — e não pretende oferecer — a extensão Timescale. A telemetria vive em hypertables com chunks de 7 dias, compressão automática após 30 dias (cerca de 90% de redução) e retenção de 5 anos. RLS funciona normalmente em hypertables, premissa verificada antes da decisão.

REGRA DE OURO DAS AGREGAÇÕES

Custo por km, consumo e rankings nunca são calculados em tempo de requisição — o alvo de 2s no p95 não sobrevive a isso. São três camadas: continuous aggregates para métricas diárias, tabelas de sumário recalculadas por evento na fila e cache Redis com TTL de 5 minutos invalidado por escrita relevante.

- Cloudflare R2 para objetos: URLs assinadas expiram em 15 minutos, o que implica reassinatura frequente — com egress cobrado, o custo seria imprevisível. O upload vai direto do cliente ao R2, sem passar pela aplicação.
- Copiloto do operador sem pipeline de vídeo: a priorização de câmeras usa sinais não-visuais (frenagem brusca, jornada, horário, histórico) e o vídeo é buscado sob demanda no fornecedor. Pipeline próprio foi formalmente rejeitado — é o maior risco de escopo do projeto.
- Dataset de treinamento: o par (frame analisado, decisão do operador) é indexado no PostgreSQL e exportado mensalmente em Parquet no R2, pronto para a Fase 2.
- Log de auditoria append-only com hash encadeado (row_hash = SHA256(prev_hash + conteúdo)): qualquer adulteração quebra a cadeia e é detectável, sem custo de infraestrutura adicional.

Convenções obrigatórias do modelo de dados

CONVENÇÃO	REGRA
Tenant	Toda tabela de domínio tem tenant_id UUID NOT NULL e policy RLS, verificados no CI
Chaves	UUID v7 — ordenável por tempo, melhor localidade de índice que o v4
Exclusão	Soft delete por deleted_at; exclusão física nunca ocorre em dado financeiro
Dinheiro	NUMERIC(18,4), nunca ponto flutuante; currency CHAR(3) DEFAULT 'BRL' desde o dia 1
Tempo	Armazenado em UTC (TIMESTAMPTZ) e apresentado em America/Sao_Paulo
Offline	filled_at (dispositivo) e received_at (servidor) em todo registro originado offline

6 Inteligência artificial

Google Gemini via API paga — a camada gratuita é proibida em qualquer ambiente que trate dado real de cliente, inclusive desenvolvimento. A orquestração usa LangChain4j, mais maduro que o Spring AI em function calling. Como já existe intenção registrada de migrar para um modelo OpenAI, a abstração LlmProvider deixa de ser boa prática e vira requisito de curto prazo: nenhuma regra de negócio pode depender de particularidade de um modelo.

O assistente não gera SQL. Ele classifica a intenção e invoca funções Java auditadas, previamente registradas, correspondentes ao catálogo de 10 intenções do PRD. Em demonstração comercial, "ainda não sei responder isso" é infinitamente preferível a um número errado — a precisão numérica é o ativo do produto.

Gate de autorização — requisito bloqueante

```
[1] classificação de intenção + function calling (mesma chamada)
[2] GATE no backend, ANTES de invocar a função:
    · o plano do tenant inclui o módulo?
    · o papel do usuário permite a ação?
    · operator_sees_financials está ativa?
    ' !O a função NÃO é chamada; recusa educada, sem revelar o dado
[3] execução determinística (j00Q, dentro de withTenant)
```

A verificação jamais ocorre por instrução no prompt: filtro por prompt não é controle de acesso. Se o dado não pode ser visto, ele não entra no contexto do modelo — a garantia é estrutural, não persuasiva. Antes do envio, o payload é tokenizado por hash determinístico com salt por tenant (nomes, placas e identificadores viram tokens opacos; CPF e telefone são removidos), e a re-hidratação acontece no backend. O mapa reverso vive apenas em memória, no escopo da requisição.

- Voz em três etapas: Google STT ! LLM com function calling ! ElevenLabs TTS, com síntese em streaming para começar a falar em menos de 6s no p95. A resposta sempre aparece também na tela.
- Toda resposta declara fonte e período ("com base em 342 abastecimentos, jan–mar/2026") e carrega ações sugeridas estruturadas, renderizadas como botões.
- Memória de 5 turnos em Redis; perguntas não atendidas são registradas e alimentam o roadmap por uso real.
- Nenhum vector database no MVP: o escopo temático é fechado e não há base documental a recuperar. Se a busca semântica se mostrar necessária, usa-se pgvector no PostgreSQL já existente.

7 Infraestrutura, integrações e segurança

INF — Computação, entrega e observabilidade

- AWS ECS Fargate na região sa-east-1: api com 2 tasks (alta disponibilidade) e worker com 1 task, ambos 1 vCPU / 2 GB. Kubernetes foi descartado — o control plane e a complexidade exigiriam quase um desenvolvedor dedicado.
- Frontends em Cloudflare Pages (painel e PWA, custo zero e egress gratuito) e Vercel (site institucional).

- Terraform/OpenTofu em vez de CDK, porque a infraestrutura não é só AWS: envolve Cloudflare, Vercel e Timescale Cloud. Migrations com Flyway em SQL puro.
- Apenas staging e production, sem ambiente de desenvolvimento compartilhado — compensado por Docker Compose local completo, Testcontainers e seeds determinísticos.
- Observabilidade sempre via OpenTelemetry, nunca pelo agente proprietário: mantém a portabilidade caso o Datadog seja trocado por Grafana Cloud.

Gates bloqueantes do pipeline

GATE	POR QUE BLOQUEIA O BUILD
tenant_id e RLS	O PRD determina explicitamente que o build falhe e que a regra não possa ser desativada por decisão de squad
Vazamento cross-tenant	Defeito que só se manifesta sob concorrência; sem teste automatizado, chega a produção
Contraste WCAG AA	Risco inerente ao glassmorphism; verificação manual não escala
Esforço do operador	Tradução mensurável do princípio inegociável do produto; regressão bloqueia o deploy

INT — Integrações com fornecedores

A camada de anticorrupção é arquiteturalmente bloqueante: sem ela, o segundo cliente com outro rastreador implicaria reescrever o núcleo. Cada fornecedor (Powerfleet, Eagletrack, Hik-Connect, TruckPag) tem um adapter que traduz para o modelo canônico da RookHub, e um teste ArchUnit garante que nenhuma regra de negócio conheça o formato de um fornecedor específico. O padrão é híbrido — webhook quando houver, polling como complemento e fallback — com retry exponencial de 1s a 32s, seis tentativas e dead letter queue com alerta. A idempotência vem da chave composta (tenant_id, provider, external_event_id).

CONFIANÇA, NÃO CONVENIÊNCIA

O banner de dado desatualizado, alimentado por last_successful_sync_at, é requisito de confiança: o dono precisa saber que está olhando um número velho antes de decidir com base nele.

Segurança e LGPD

- TLS 1.2+ terminado no ALB, criptografia em repouso, Argon2id para senhas e PINs, rate limiting por tenant e por usuário, URLs assinadas com no máximo 15 minutos.
- Segredos exclusivamente no AWS Secrets Manager — nunca em variável de ambiente de imagem ou em repositório. Imagens escaneadas por Trivy, base distroless, CSP restritiva e IAM com menor privilégio.
- A aplicação conecta ao banco com usuário sem BYPASSRLS.
- LGPD: base legal documentada, termo de ciência registrado no primeiro acesso do motorista, jobs de expurgo por categoria, anonimização de motorista desligado há 12+ meses preservando as métricas agregadas e tokenização antes de qualquer envio à IA.
- O documento não constitui aconselhamento jurídico: RIPD, política de privacidade e termo de ciência precisam de validação por advogado antes do go-live.

8 Custos e riscos

ITEM	FORNECEDOR	USD/MÊS
Banco de dados	Timescale Cloud	50 – 150
Computação (api + worker)	AWS ECS Fargate	60 – 100
Load balancer e cache/filas	AWS ALB + ElastiCache	35 – 50
Object storage (~500 GB) e frontends	Cloudflare R2 + Pages/Vercel	8 – 28
LLM, STT e TTS	Gemini + Google STT + ElevenLabs	124 – 250

ITEM	FORNECEDOR	USD/MÊS
Observabilidade e rastreo de erros	Datadog + Sentry	176 – 276
E-mail, push, ligações, mapas e diversos	SES, FCM, Zenvia, Mapbox, AWS	21 – 115
Total estimado — 2 clientes, ~80 veículos		"H 475 – 970

Os dois maiores itens da fatura — Datadog e ElevenLabs, somando US\$250 a 400 — não são infraestrutura de produto, mas escolhas de qualidade. Ambos têm gatilho de reavaliação documentado e alternativas cerca de 10× mais baratas: trocando por Grafana Cloud e Google TTS, o total cairia para algo em torno de US\$250 a 450 por mês. A projeção para 10 clientes e cerca de 800 veículos fica entre US\$1.200 e 2.000 — o custo por cliente cai com a escala, característica saudável do modelo.

Principais riscos e dívidas assumidas

ID	RISCO OU DÍVIDA	MITIGAÇÃO
RT-01	Limitações de PWA em iOS (push, cota de IndexedDB, expurgo do Safari)	Levantar a distribuição Android/iOS da frota-âncora antes da sprint 1; fallback por SMS
RT-02	Custo do Mapbox escalando de forma não-linear	Instância única de mapa, atualização por setData e migração para MapLibre acima de US\$100/mês
RT-03	Pressão futura por pipeline de vídeo próprio	Formalmente rejeitado — transformaria a RookHub em empresa de infraestrutura de mídia
RT-04	TTS ElevenLabs custa ~10× as alternativas	Cache de respostas frequentes, abstração de provedor e gatilho de migração em 15% da fatura
RT-06	Datadog custa mais que o banco de dados	Plano Pro sem RUM, amostragem de traces, retenção de 7 dias e portabilidade via OpenTelemetry
RT-09	Dependência de fornecedores de telemetria para a proposta de valor	ACL obrigatória, banner de dado desatualizado e monitoramento do last_successful_sync_at
DT-01	Kafka adiado para a Fase 2	Camada de filas abstraída por interface desde o MVP
DT-03	Migração planejada de Gemini para OpenAI	Abstração LlmProvider desde o dia 1

9 Onde o projeto está hoje — Fase 1

O repositório atual entrega o MVP frontend do painel web: todas as telas dos módulos do produto, com dados simulados e arquitetura preparada para trocar os mocks por HTTP sem reescrever tela alguma. Não existe backend, banco, autenticação real, telemetria real nem chamada a modelo de IA. A tabela abaixo confronta o alvo do documento com o que está implementado.

CAMADA	DOCUMENTO (ALVO)	IMPLEMENTADO HOJE
Aplicações	Monorepo Turborepo/pnpm com apps/web, apps/driver e apps/site	Repositório único com o painel web; PWA do motorista e site institucional não iniciados
Framework	Vite + React 19 + React Router	Vite 8 + React 19.2 + React Router 8, rotas públicas, protegidas e admin com lazy loading
Estilização	Tailwind v4 + shadcn/ui + tokens	Igual — Tailwind 4 CSS-first (@theme em globals.css), shadcn/ui sobre Radix
Tokens	Rampa OKLCH sobre Indigo, Cyan e Midnight	Igual — CSS variables em OKLCH, tema escuro padrão e tema claro alternável
Tipografia e ícones	Sora + Inter, Phosphor Icons	Space Grotesk + Inter, ícones lucide-react

CAMADA	DOCUMENTO (ALVO)	IMPLEMENTADO HOJE
Estética	Glassmorphism total	Corporativa e densa em informação, sem neon nem glow exagerado
Estado	TanStack Query + Zustand	Igual, mais react-hook-form + Zod 4 nos formulários
Gráficos	visx no Painel do Dono + Recharts nas demais telas	Somente Recharts, por meio de um módulo de gráficos compartilhado
Mapas	Mapbox GL JS, com migração prevista acima de US\$100/mês	MapLibre GL + OpenFreeMap — a mitigação do RT-02 já foi adotada: sem chave, sem custo por map load
Tempo real	WebSocket STOMP + Redis pub/sub	Não implementado — depende do backend
Backend e banco	Java 21 + Spring Boot/Modulith sobre PostgreSQL + TimescaleDB com RLS	Inexistente. Os dados vêm de mocks servidos por uma camada de serviços com atraso de rede simulado
Autenticação	Spring Security, JWT + refresh rotativo, Argon2id	Sessão simulada em store Zustand; nenhum token real é gerado ou persistido
Permissões e planos	Entitlement e RBAC verificados no backend	Guardas de permissão e de plano no frontend — assumidamente UX, não segurança
IA	LangChain4j + Gemini com function calling e gate de autorização	Nenhuma chamada a LLM; textos demonstrativos. A chave do Gemini está reservada ao backend futuro
Voz	Google STT + ElevenLabs TTS em streaming	TTS ElevenLabs já funciona por proxy no servidor de desenvolvimento, com a chave lida apenas no Node; STT ainda não
Visual 3D	Não previsto no documento	Three.js na esfera de partículas do assistente, no globo do painel e no vórtice da tela de sessão expirada

Princípios do documento que já regem o código

- Nenhum segredo no frontend: só variáveis VITE_* chegam ao bundle, e as chaves de IA e de voz são lidas exclusivamente no processo Node.
- As telas consomem a camada de serviços, nunca os mocks diretamente — é o que permitirá trocar cada implementação por uma chamada HTTP sem tocar na interface.
- As guardas de permissão e de plano estão documentadas como UX; a validação definitiva é responsabilidade do backend.
- Tabela única compartilhada, mocks fora dos componentes e tokens de cor sem valores literais repetidos.
- Bateria de verificação obrigatória antes de cada marco: formatação, typecheck, lint, testes e build.

10 Próximos passos

Bloqueantes antes da sprint 1 (documento)

- Levantar a distribuição Android/iOS da frota-âncora — pode alterar a estratégia do aplicativo do motorista.
- Validar o meio físico do alerta em cabine antes de qualquer sprint de hardware.
- Obter a documentação de API dos quatro fornecedores do MVP — a modelagem canônica depende do formato real.
- Contratar a validação jurídica do RIPD e da política de privacidade.

Sprint 0 — fundação técnica

- Monorepo, Terraform e pipelines de CI/CD.
- Esqueleto multi-tenant com RLS e os quatro gates de garantia funcionando — antes de qualquer regra de negócio.
- Pacotes de tokens e de UI com os primitivos de vidro e o fallback de blur.
- Autenticação completa (os três fluxos) e estrutura de módulos do Modulith com testes de fronteira.

Pendências do frontend atual

- Substituir as implementações mockadas por chamadas HTTP reais e adotar paginação server-side.
- Autenticação real com token e refresh, e revalidação de permissão e plano no backend.
- Provedores reais de mapa, telemetria, rastreamento, multas e IA.
- Smoke test end-to-end e code splitting dos chunks que ultrapassam 500 kB.

A RECOMENDAÇÃO MAIS IMPORTANTE DO DOCUMENTO

O isolamento multi-tenant deve estar completo, testado e blindado por CI antes da primeira regra de negócio ser escrita. Retrofit de RLS em base já povoada é a operação mais cara e arriscada desta arquitetura.